

Real-time Pacing



<https://forms.gle/u8FR8MZg8sLPVrmM8>

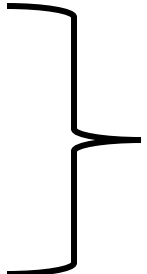
Leading Questions

How to construct a real-world CPA-secure encryption scheme from block cipher?

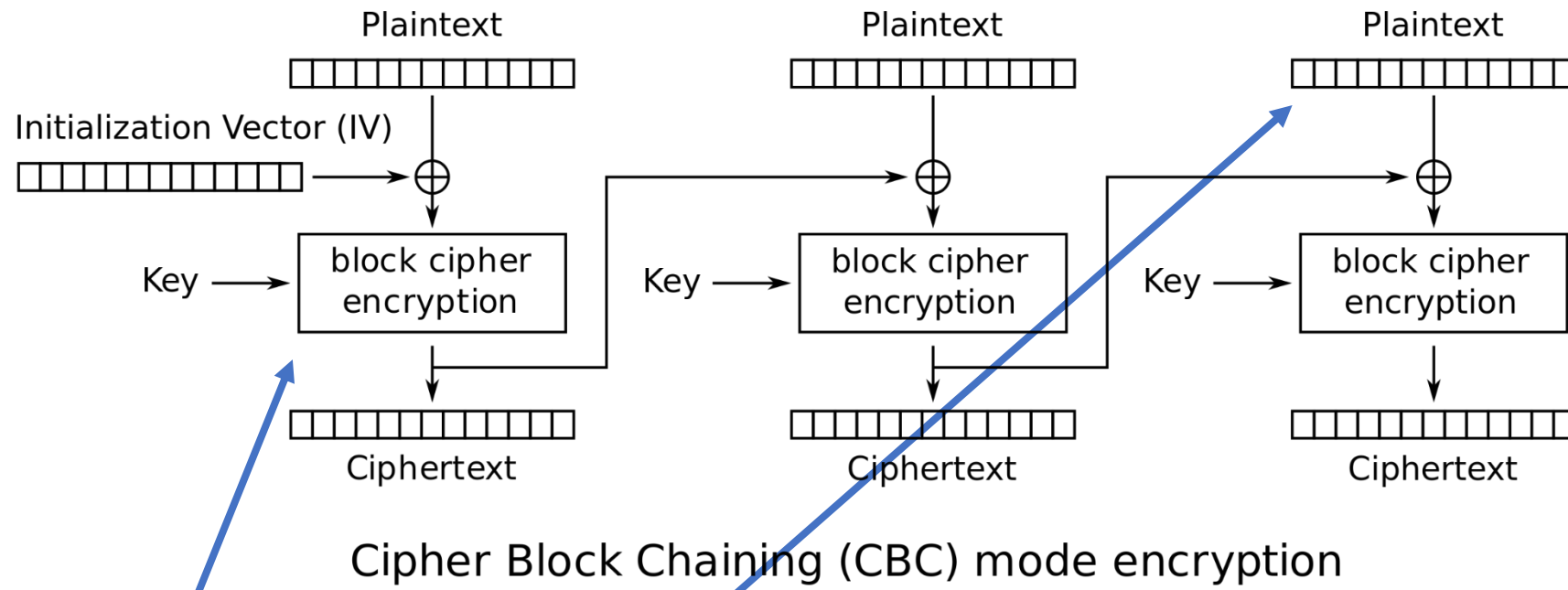
1. What are the constructions of block cipher?
2. What are the encryption modes?
3. Security/**Attacks?**

Modes of encryption

× ECB mode

- CBC mode
 - OFB mode
 - CTR mode
- 
- CPA secure

Recall



1. Block cipher
2. Mode of encryption
3. Padding

Padding

- Pad (encode) the message so that the length is a multiple of n
- Receiver can unambiguously recover the message (decode)
- Two types of padding functions
 - Bit-oriented, pad 100000.... see [ISO/IEC 9797-1](#)
 - Byte-oriented, message consists of an integral number of bytes

Padding

- Example: PKCS#7 is a byte-oriented padding function

Algorithm CBCPAD(m):

let $m_1 | \dots | m_l = m : m_i \in \{0,1\}^8$

$p \leftarrow n - (l \bmod n)$

return $m | \underbrace{pp \dots p}_{p \text{ times}}$

n : **block size in bytes** of
the cipher (eg. $n=16$ for
AES, $n=8$ for DES)

■ Padding is “01”, “02 02”, “03 03 03”, “04 04 04 04” ...

... | DD DD DD DD DD DD DD DD | DD DD DD DD 04 04 04 04 |

▼ Denote this by “ $p \times p$ ”

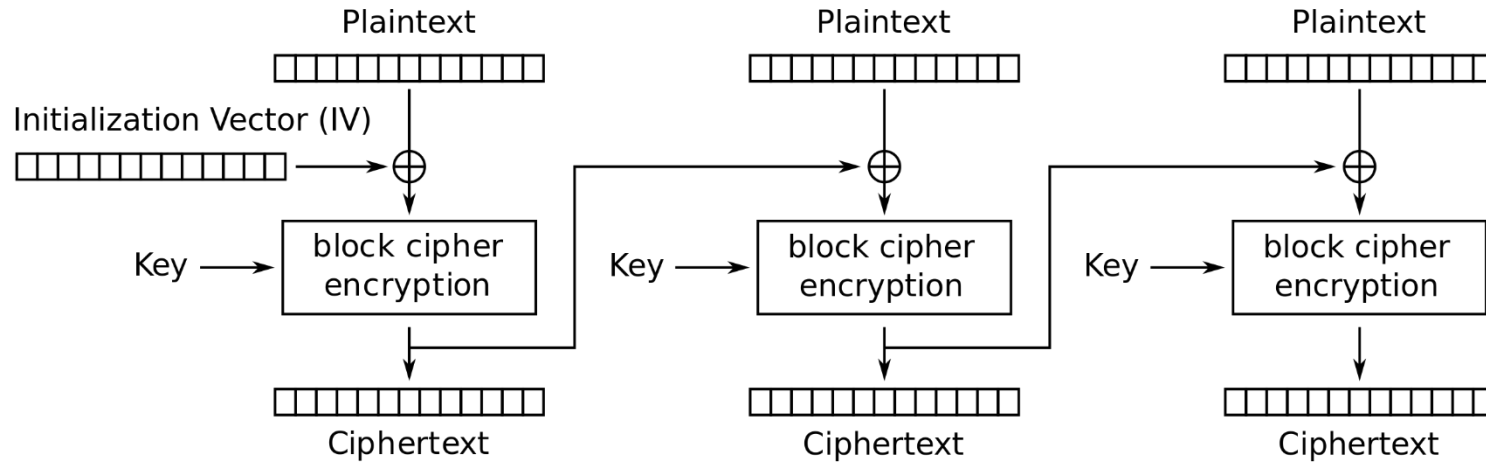
Q: If m is a multiple of $n=16$ bytes (128 bit)

A: pad 16×16

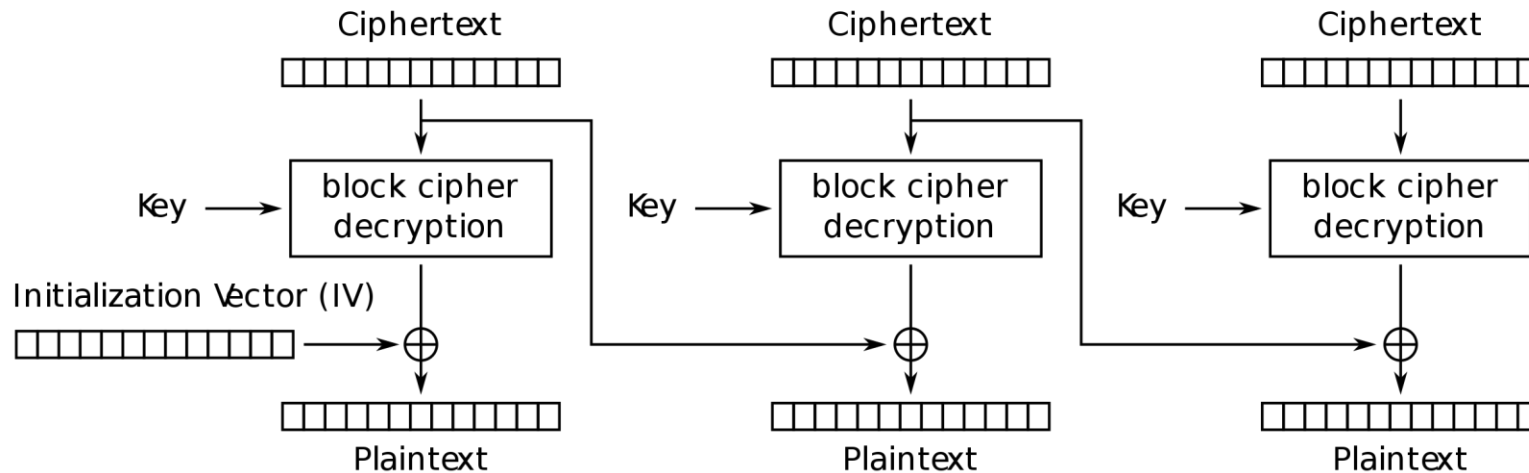
Padding Oracle $P_k(c) \rightarrow 0 \text{ or } 1$

- Oracle: black-box responder
- Padding Oracle: respond if padding is valid
- What if the padding in a ciphertext is not valid?
 - tear down the session (as in SSL/TLS)?
 - log the error (as in ESP (part of IPSec), web oracle expose)?
 - return an error message (as in WTLS)?
- Either way, typically will leak **whether the plaintext padding was valid**
- Abstract this as an oracle $P_k(c) \rightarrow 0 \text{ or } 1$

Padding Oracle Attack Example: CBC



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

Enc and Dec of CBC

Let $f: \text{Key} \times \{0,1\}^n \rightarrow \{0,1\}^n$ be a block cipher (e.g. AES)

Algorithm $\text{Enc}_K(m)$:

let $m_1 | \dots | m_l = m : m_i \in \{0,1\}^n$

$c_0 \leftarrow_R \{0,1\}^n$

for $i = 1 \dots l$ do $c_i \leftarrow f_K(c_{i-1} \oplus m_i)$

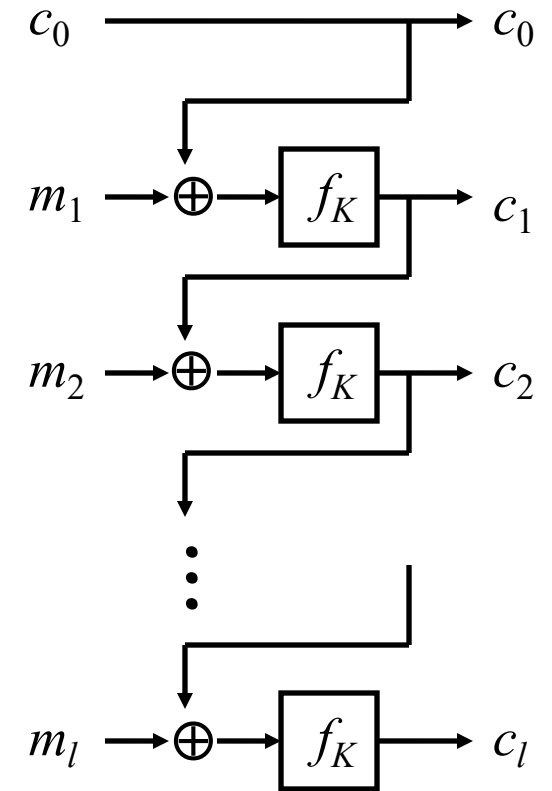
return $c_0 | c_1 | \dots | c_l$

Algorithm $\text{Dec}_K(c)$:

let $c_0 | c_1 | \dots | c_l = c : c_i \in \{0,1\}^n$

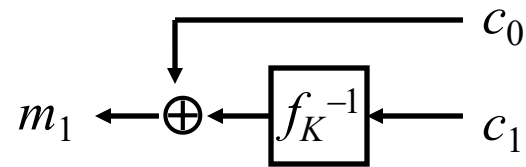
for $i = 1 \dots l$ do $m_i \leftarrow f_K^{-1}(c_i) \oplus c_{i-1}$

return $m_1 | \dots | m_l$



Last Byte Decryption [Vaudenay 2002]

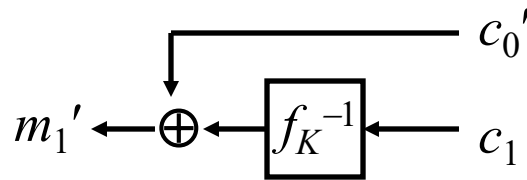
- Consider a two-block ciphertext $c_0 \mid c_1$
- We know that decryption is performed as follows



$$c_0 \oplus f_K^{-1}(c_1) = m_1$$

$$\Downarrow \\ f_K^{-1}(c_1) = m_1 \oplus c_0$$

- Attacker can choose $c_0' \neq c_0$



$$c_0' \oplus f_K^{-1}(c_1) = m_1'$$

$$\Downarrow \\ f_K^{-1}(c_1) = m_1' \oplus c_0'$$

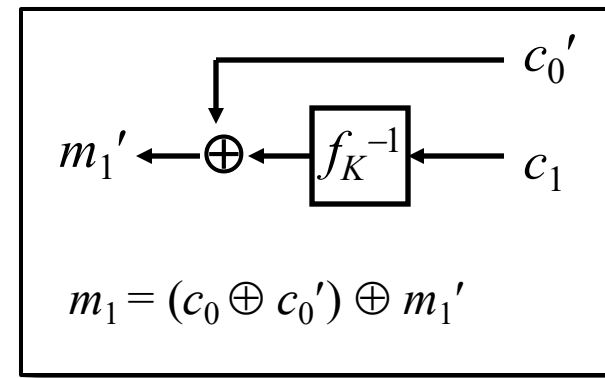
- Since $c_0 \oplus m_1 = f_K^{-1}(c_1)$ and $c_0' \oplus m_1' = f_K^{-1}(c_1)$

we get $c_0 \oplus m_1 = c_0' \oplus m_1'$

we get $m_1 = (c_0 \oplus c_0') \oplus m_1'$

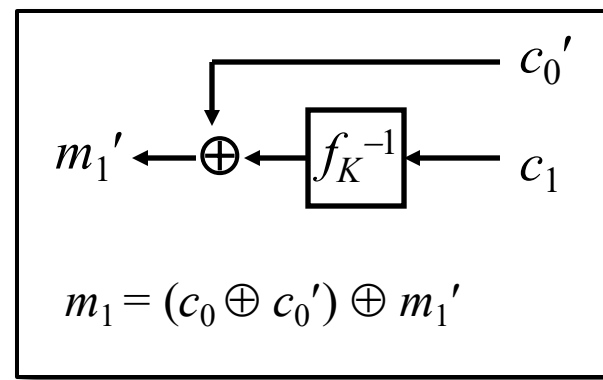
- What does attacker know here? Attacker knows $(c_0 \oplus c_0')$ but not m_1'

Last Byte Decryption (cont.)



- However, if attacker can find c_0' such that $P_K(c_0' | c_1) = 1$, then attacker know that m_1' is correctly padded
- $m_1 = (c_0 \oplus c_0') \oplus m_1'$
- So, if m_1' is correctly padded, we could just assume that m_1' ends in 01
 - If correct, then last byte of m_1 is last byte of $(c_0 \oplus c_0') \oplus 01$

Last Byte Decryption (cont.)



- Given $P_K(c_0' | c_1) = 1$,
(m_1' is correctly padded), the attacker can change c_0' to figure out the exact padding of m_1'

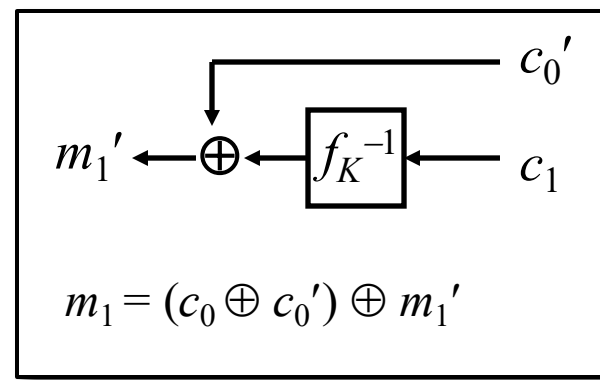
$$m_1' = c_0' \oplus f_K^{-1}(c_1)$$

Bit flipping

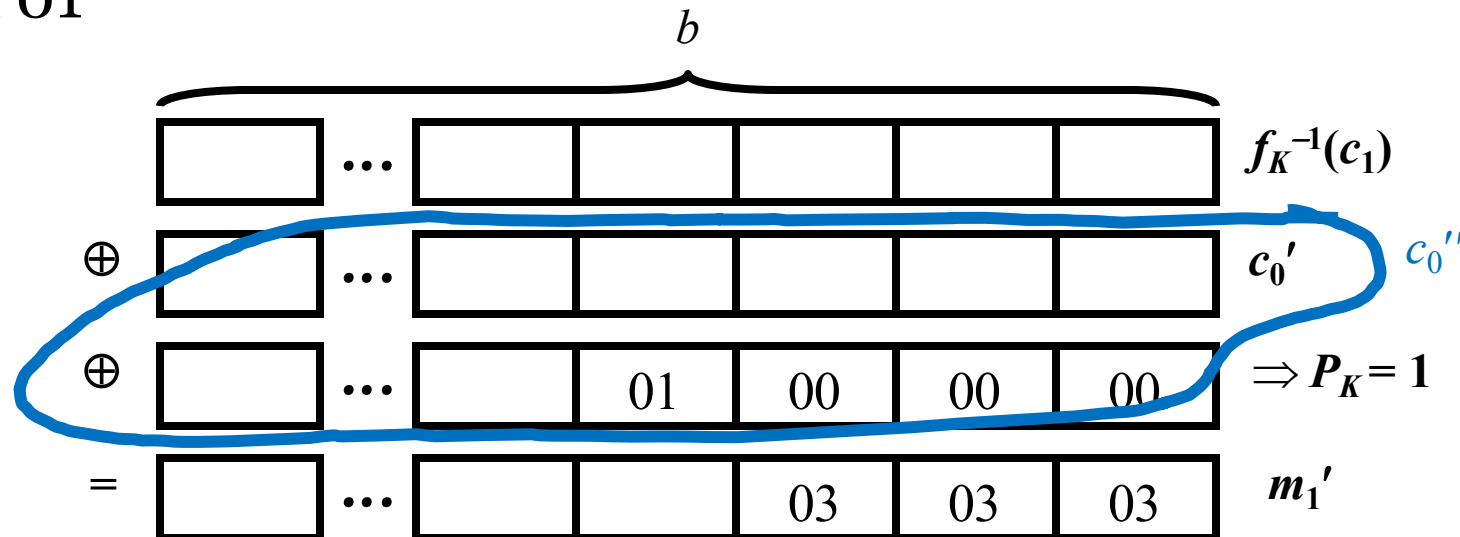
m_1' ends in 01 or 02 02 or 03 03 03 or

Last Byte Decryption (cont.)

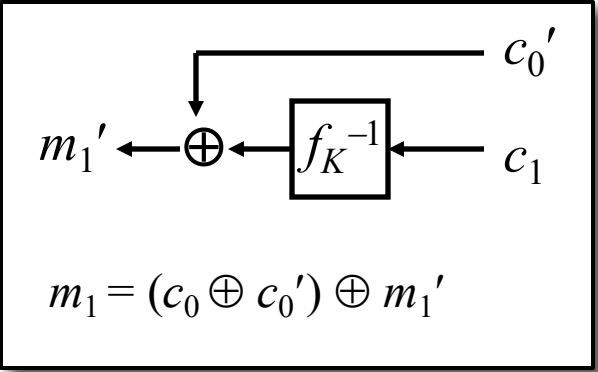
- To get it right in all cases, start from c_0' where $P_K(c_0' | c_1) = 1$ and do the following



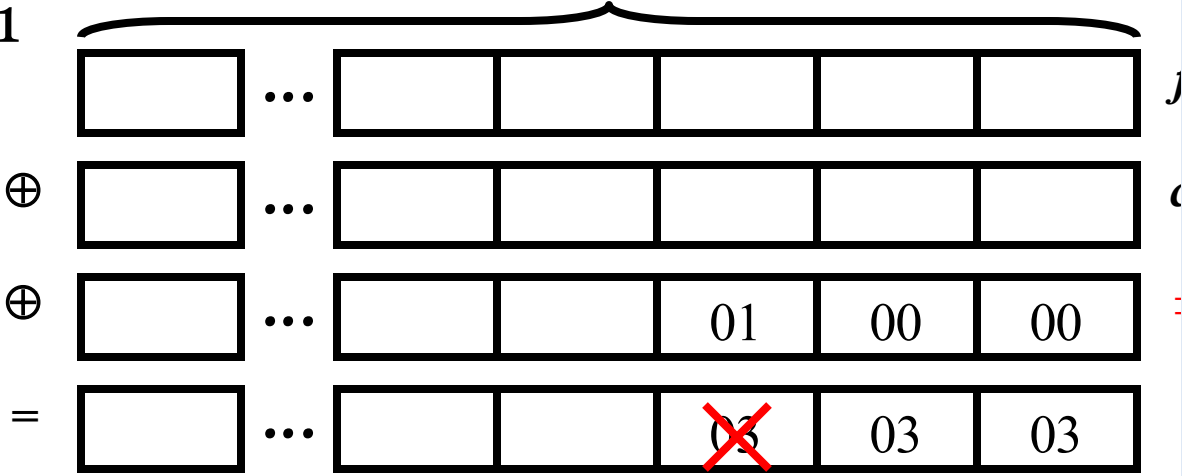
- If $P_K((c_0' \oplus 01(00)^{b-1}) | c_1) = 0$ then m_1' ends in $b \times b$, else
- If $P_K((c_0' \oplus 01(00)^{b-2}) | c_1) = 0$ then m_1' ends in $b-1 \times b-1$, else
- ...
- If $P_K((c_0' \oplus 01(00)^1) | c_1) = 0$ then m_1' ends in 02 02, else
- m_1' ends in 01



Last Byte Decryption (cont.)

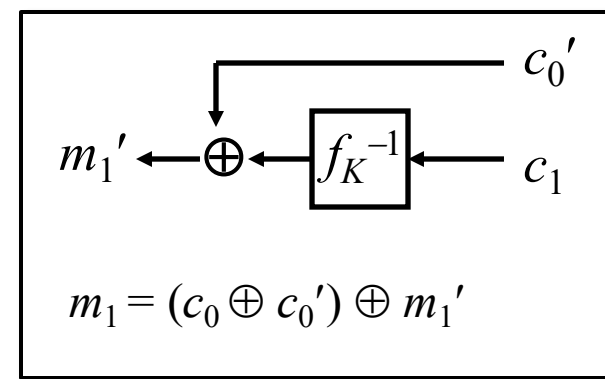


- To get it right in all cases, start from c_0' where $P_K(c_0' | c_1) = 1$ and do the following
 - If $P_K((c_0' \oplus 01(00)^{b-1}) | c_1) = 0$ then m_1' ends in $b \times b$, else
 - If $P_K((c_0' \oplus 01(00)^{b-2}) | c_1) = 0$ then m_1' ends in $b-1 \times b-1$, else
 - ...
 - If $P_K((c_0' \oplus 01(00)^1) | c_1) = 0$ then $m_1 b$ ends in 02 02, else
 - m_1' ends in 01



Block Decryption

- Now we can use this to find all of m_1



$$\oplus \begin{array}{c} \boxed{} \dots \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} f_K^{-1}(c_1) \\ \boxed{} \dots \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} c_0' \end{array}$$

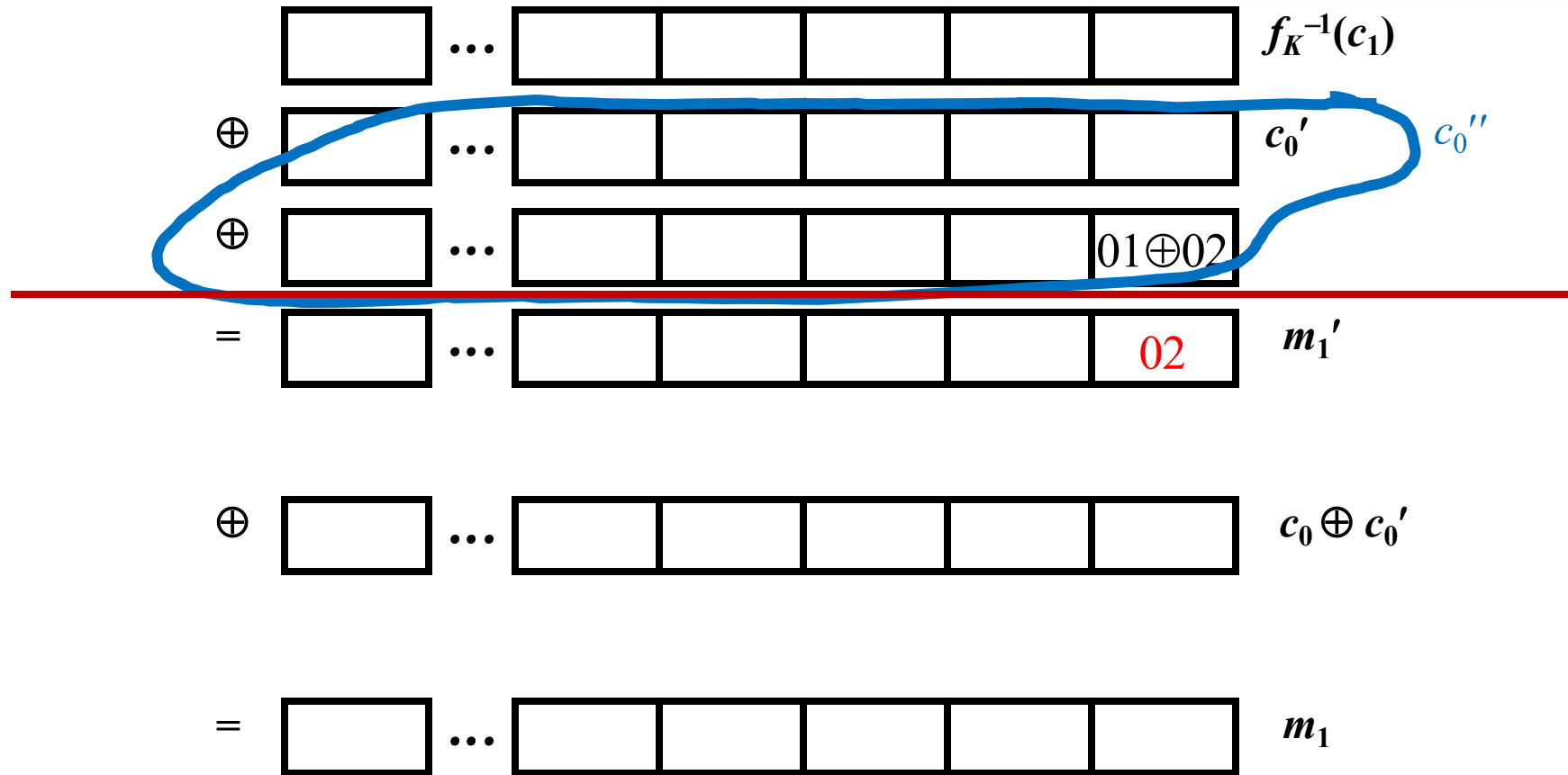
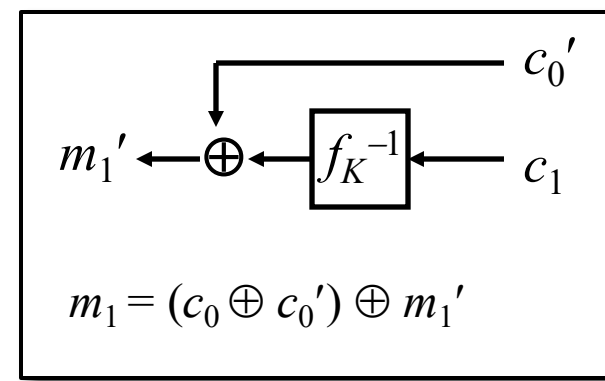
$$= \boxed{} \dots \boxed{} \boxed{} \boxed{} \boxed{} \boxed{01} m_1'$$

$$\oplus \boxed{} \dots \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} c_0 \oplus c_0'$$

$$= \boxed{} \dots \boxed{} \boxed{} \boxed{} \boxed{\text{✓}} m_1$$

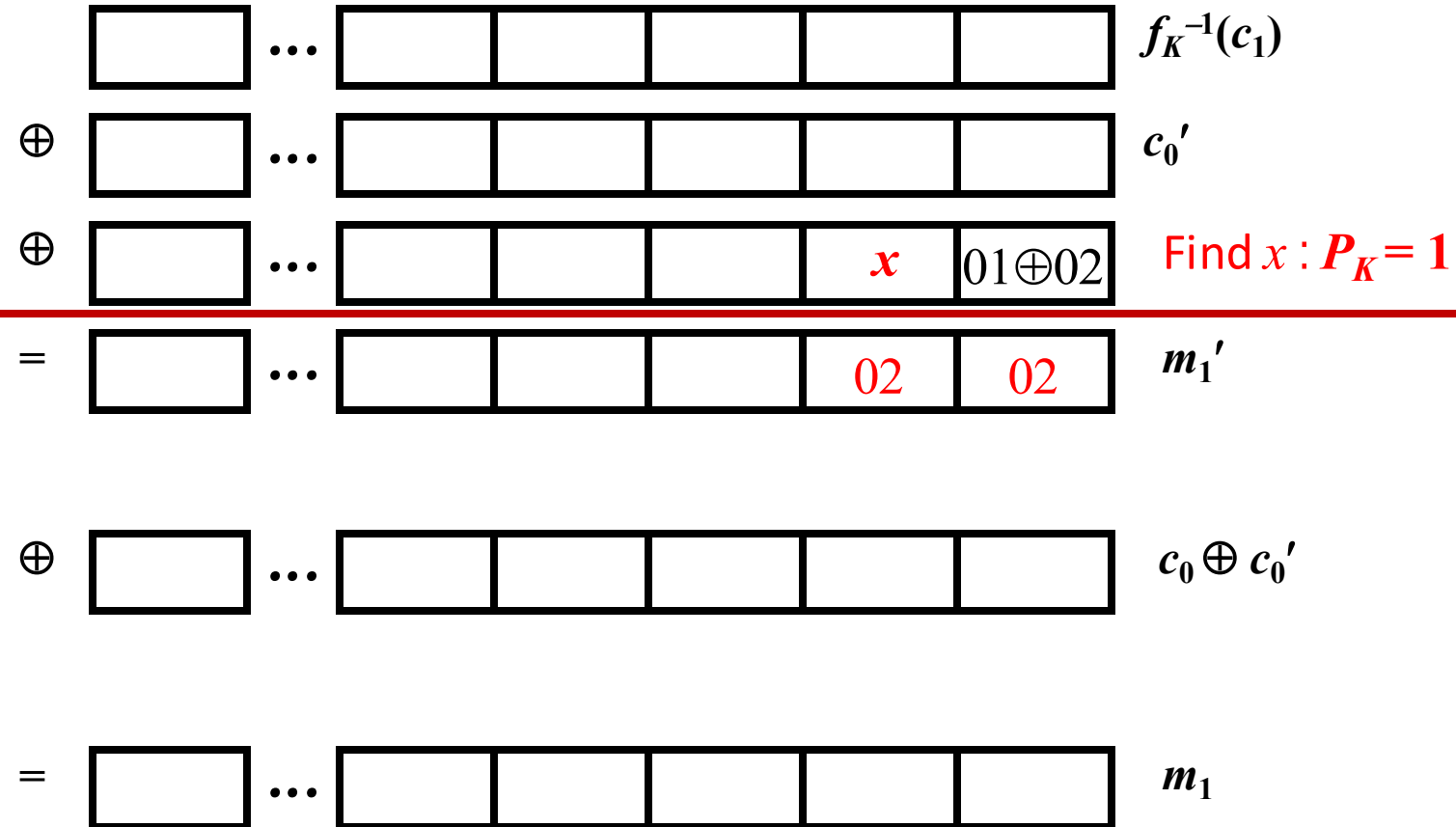
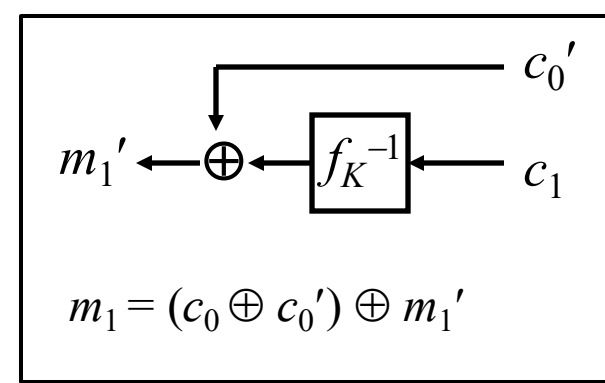
Block Decryption

- Now we can use this to find all of m_1



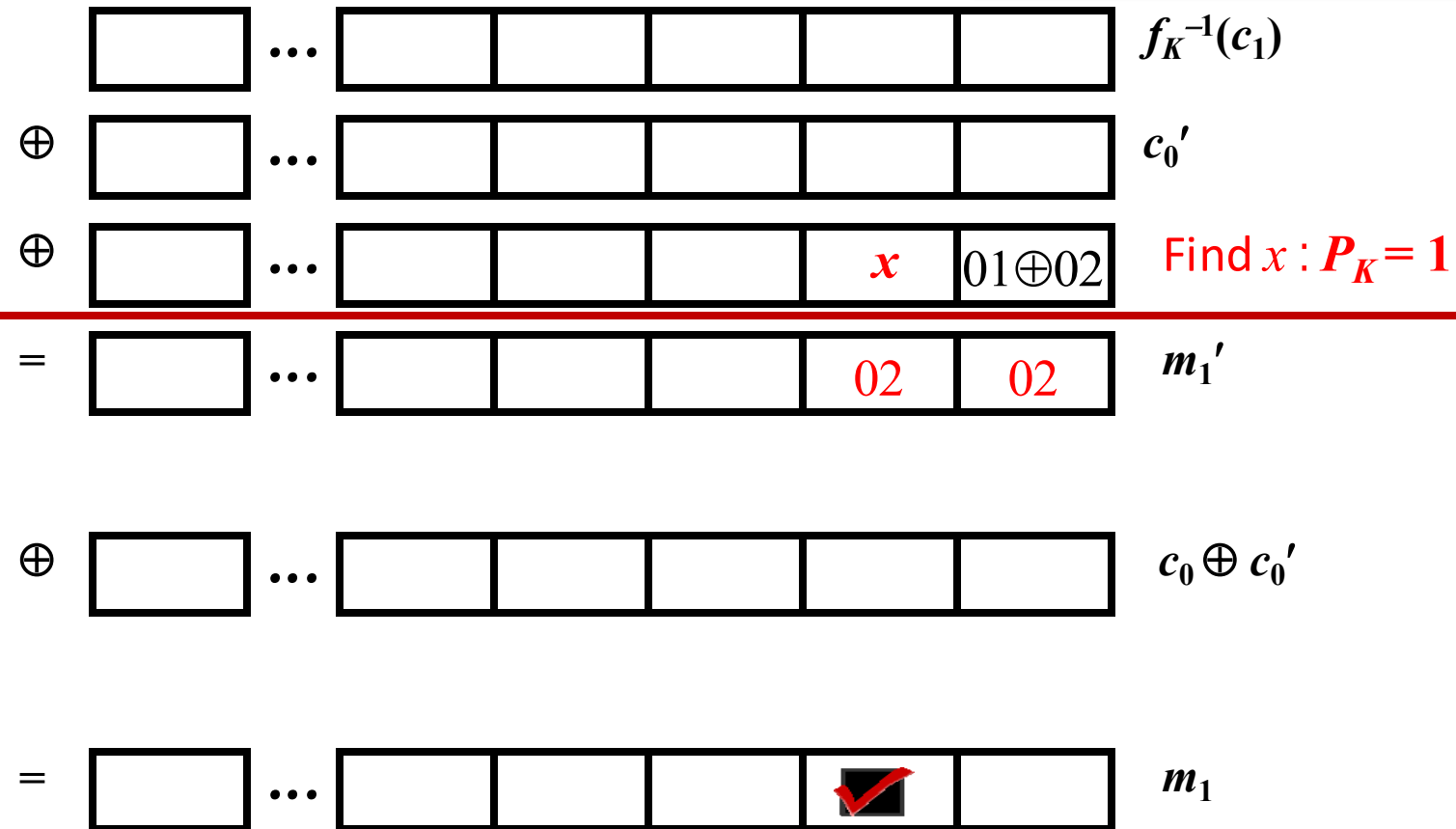
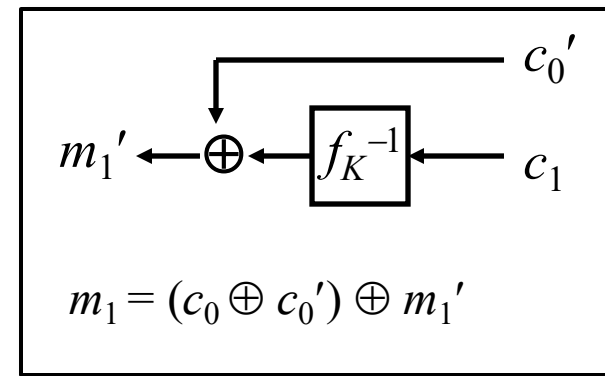
Block Decryption

- Now we can use this to find all of m_1



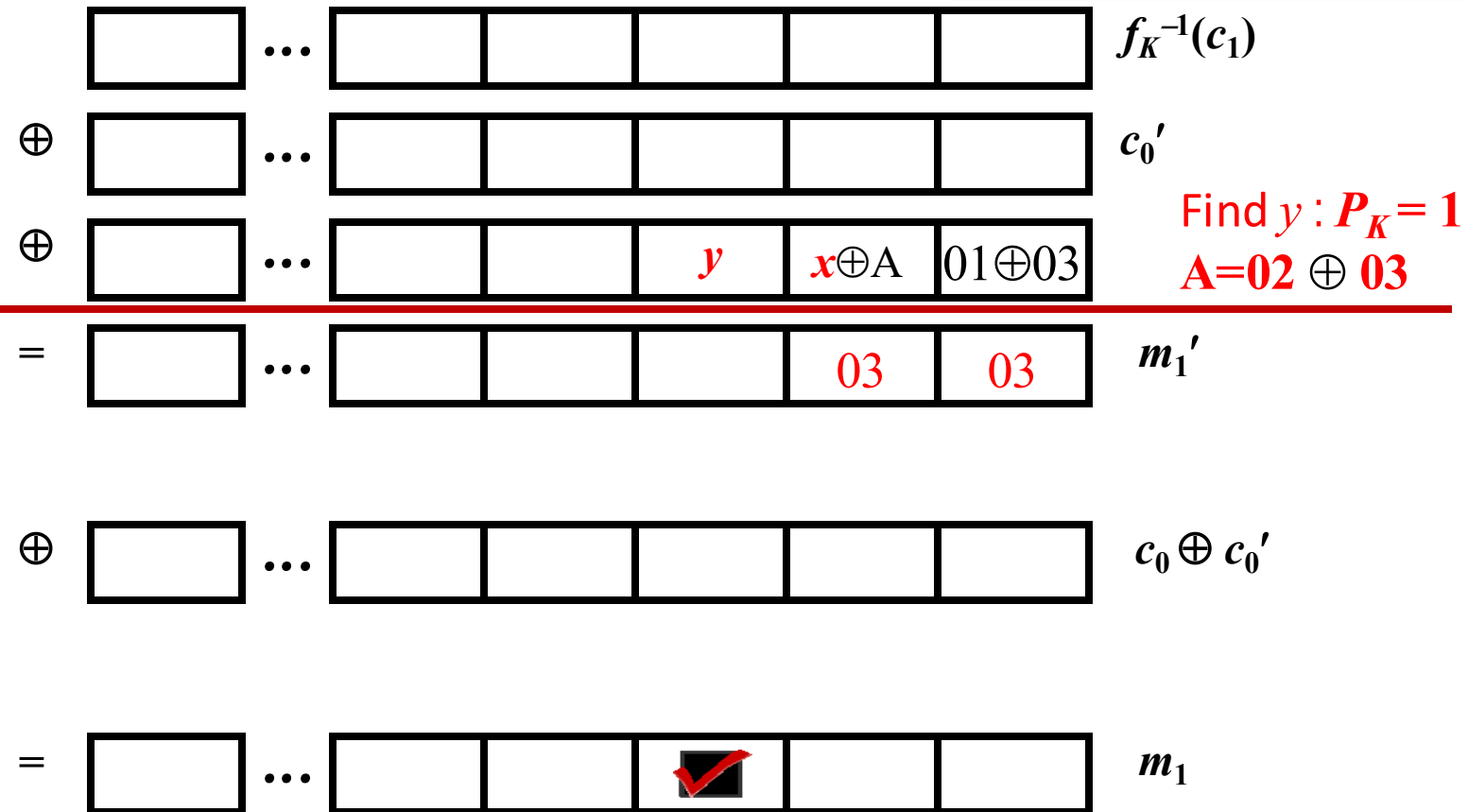
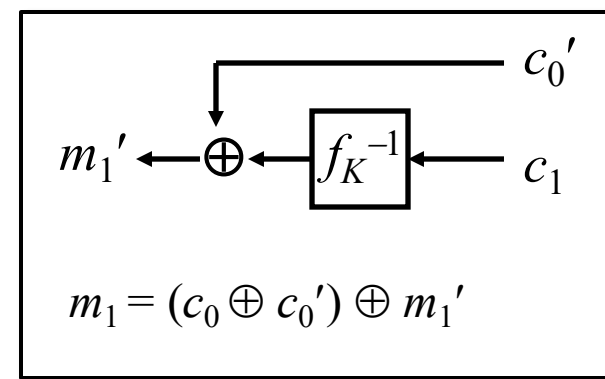
Block Decryption

- Now we can use this to find all of m_1



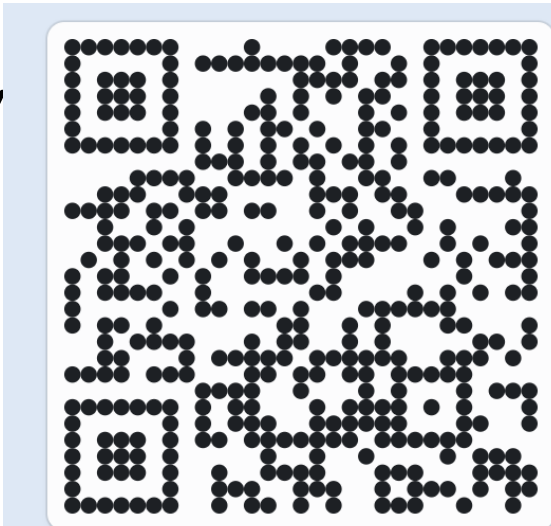
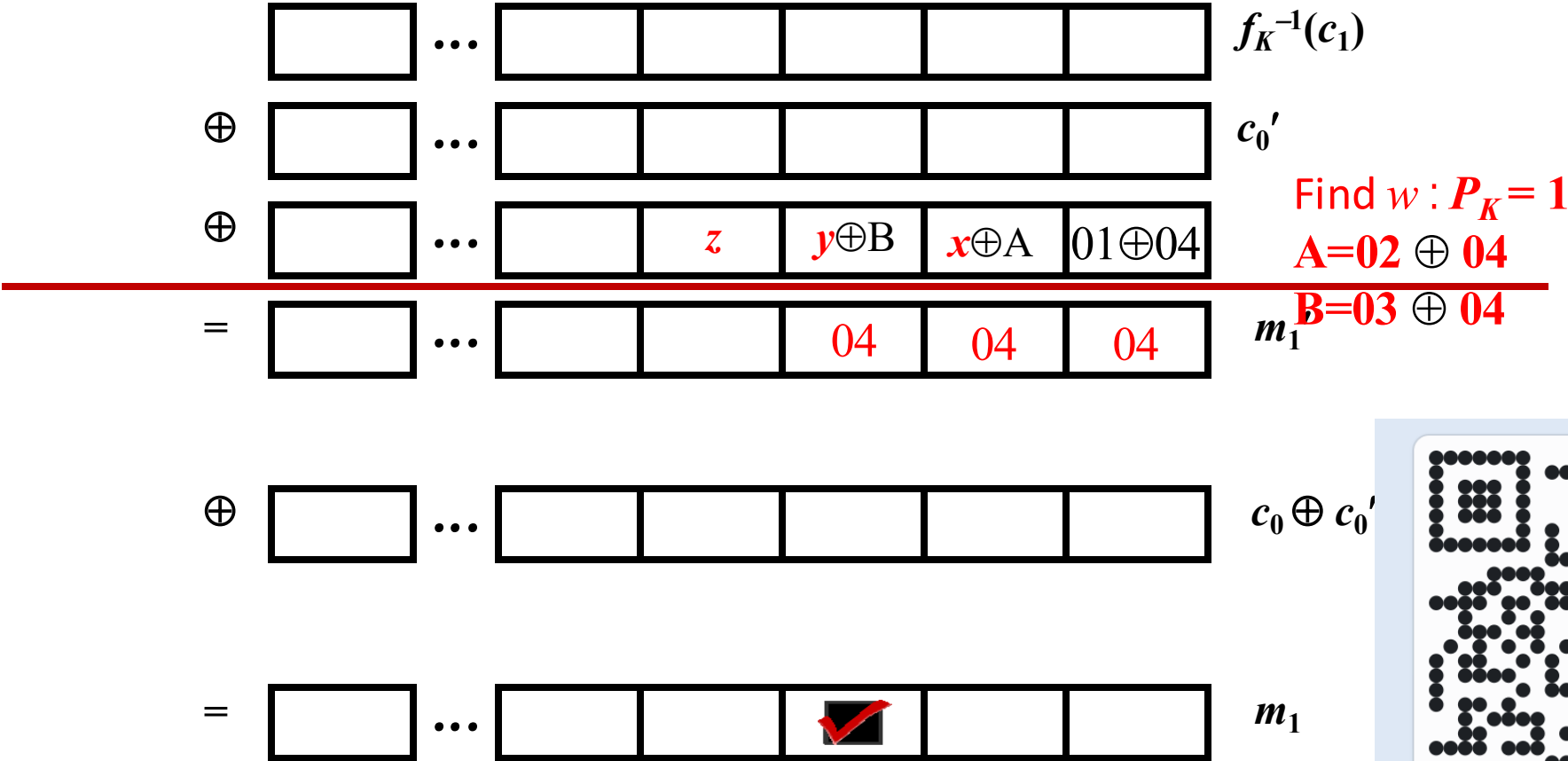
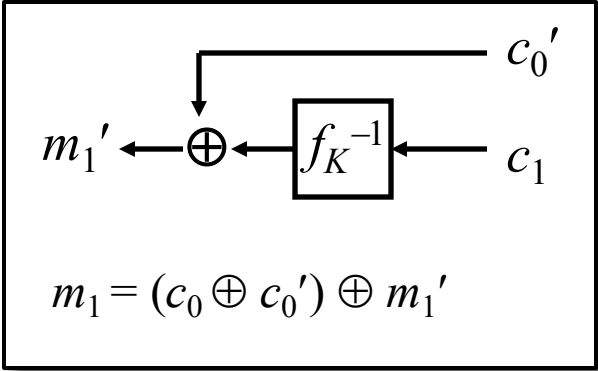
Block Decryption

- Now we can use this to find all of m_1

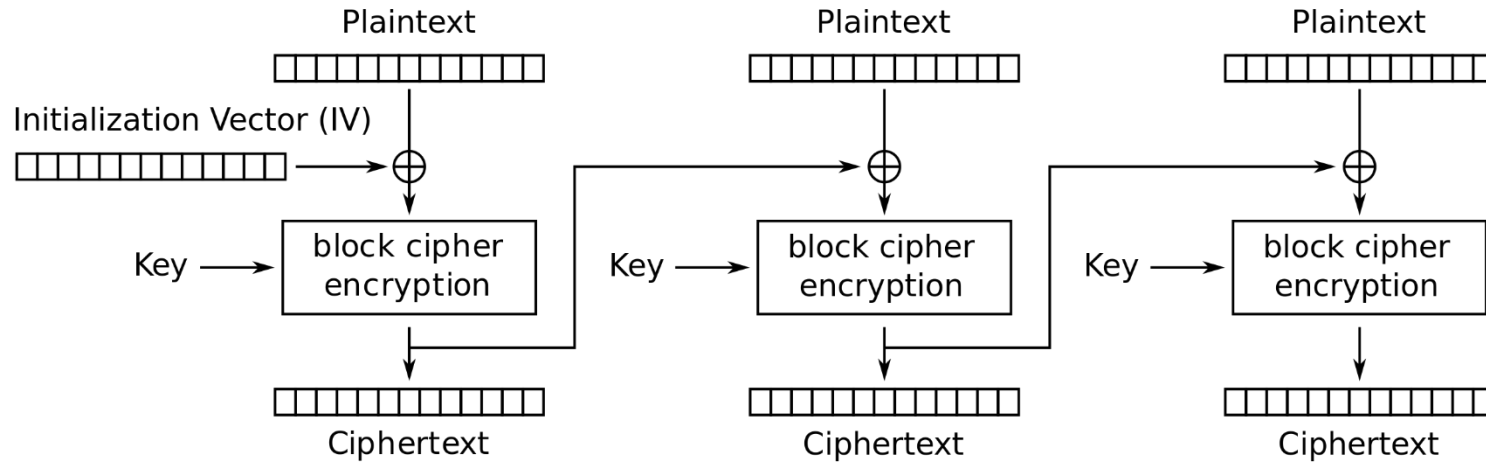


Block Decryption

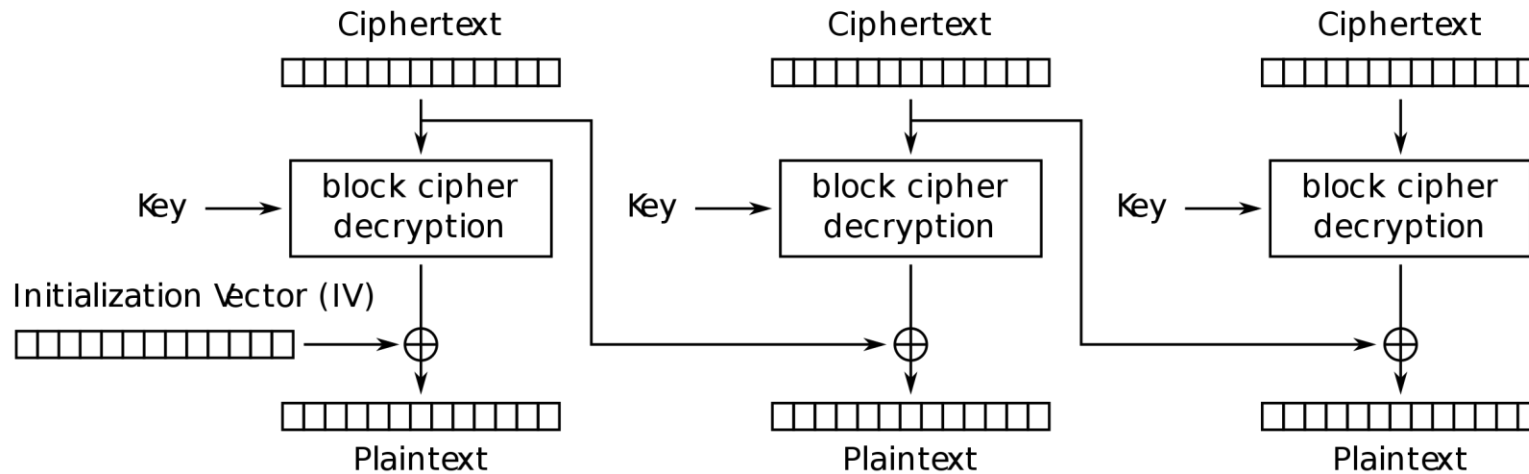
- Now we can use this to find all of m_1



CBC Mode



Cipher Block Chaining (CBC) mode encryption



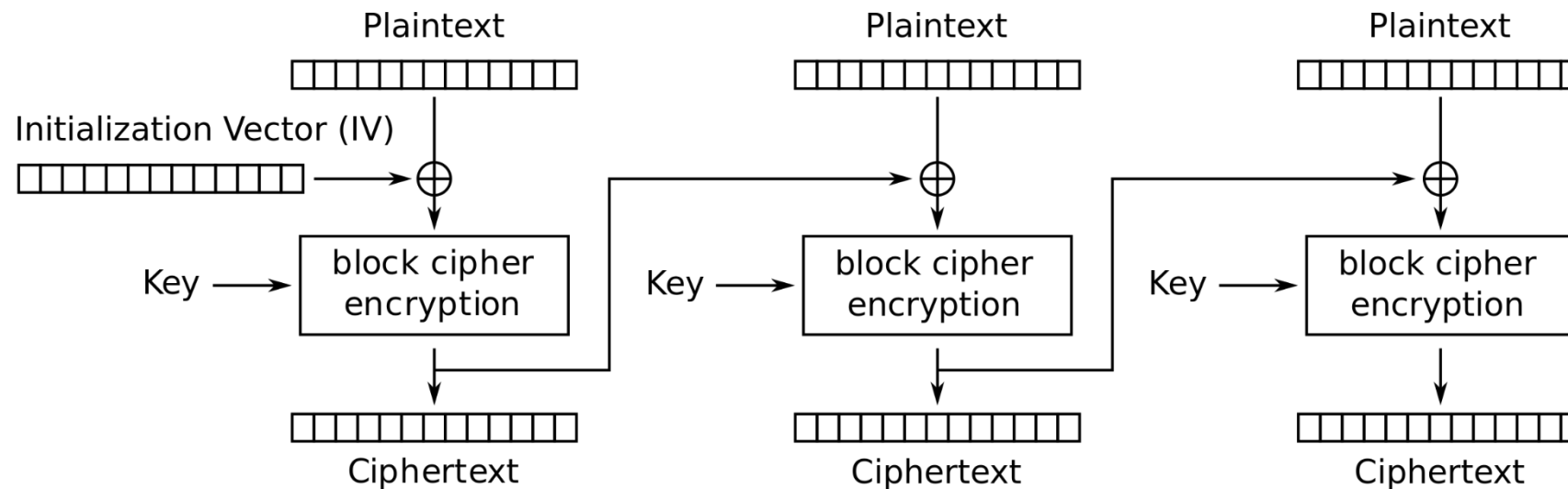
Cipher Block Chaining (CBC) mode decryption

Full Decryption

- Once we've implemented block decryption, full decryption of multi-block messages is straightforward
 - Do each block separately
 - Use preceding ciphertext block as its initialization vector
- Block decryption can be sped up using binary search instead of linear search to find padding length

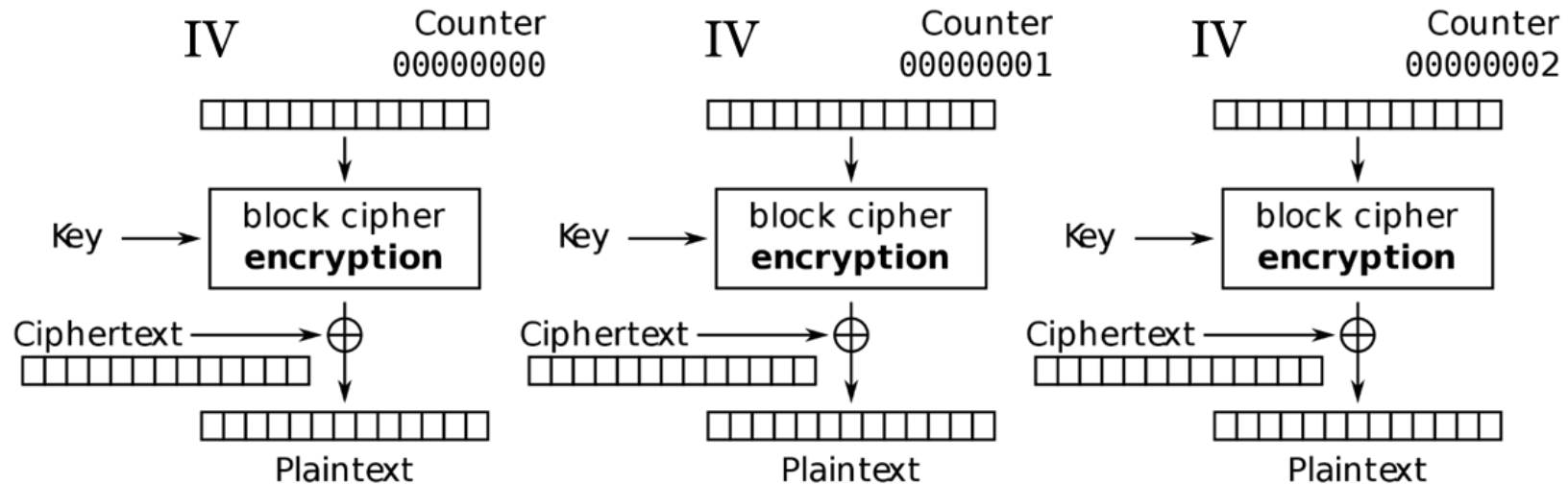
Full decryption

- Last 2 blocks form a valid ciphertext
- Decrypt block by block (depending on the rules)



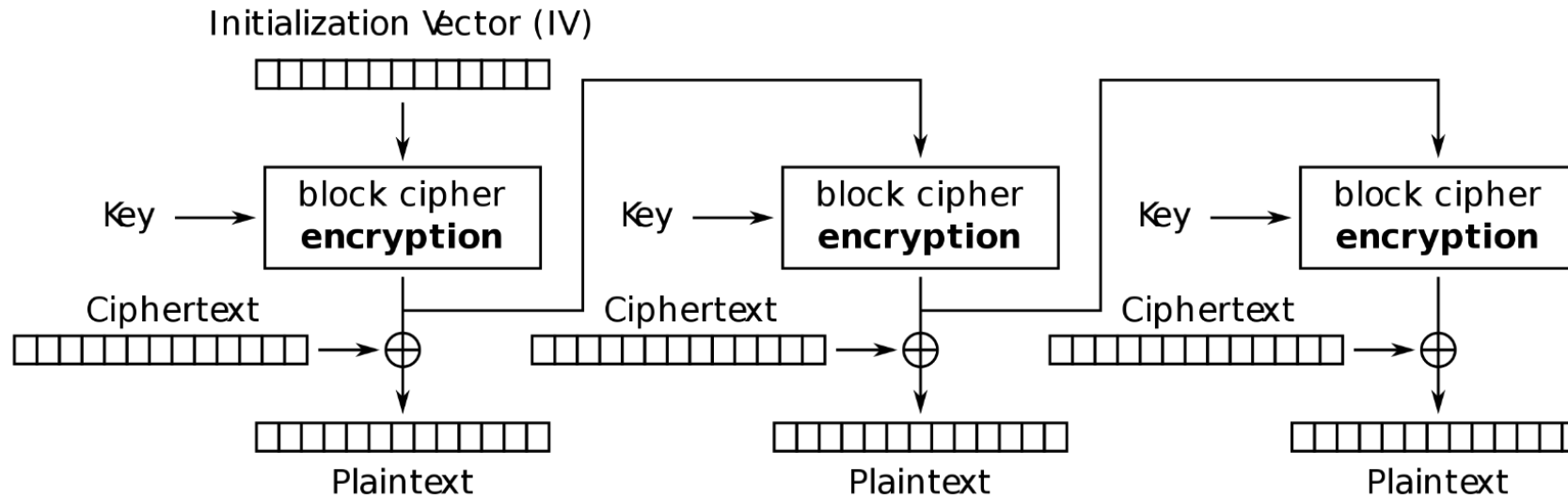
Cipher Block Chaining (CBC) mode encryption

Not unique to CBC



Counter (CTR) mode decryption

Not unique to CBC



Output Feedback (OFB) mode decryption

Why is padding oracle attack possible?

- Problem: chosen-plaintext (CPA) security is not strong enough
- Solution: chosen-ciphertext (CCA) security and authenticated encryption
 - Message authentication code (MAC)
 - Cannot change ciphertext and number of blocks

Chosen-plaintext attack (CPA) security

Experiment $\text{PrivK}_{A,\Pi}^{\text{CPA}}(n)$:

1. A key k is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , and **oracle access to $\text{Enc}_k(\cdot)$** . A outputs two messages $m_0, m_1 \in \mathcal{M}$ s.t. $|m_0| = |m_1|$
3. A random bit $b \in \{0,1\}$ is chosen. $c = \text{Enc}_k(m_b)$ is given to A .
4. A continues to have **oracle access to $\text{Enc}_k(\cdot)$** and outputs b'
5. Experiment outputs 1 if $b=b'$, and 0 otherwise.

Definition: a private-key encryption $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is CPA-secure, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{PrivK}_{A,\Pi}^{\text{CPA}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$